

ORACLE

23c New Features for XML: Sharding of XML & XML Search Index

Technical Overview



Agenda

- **What is Database Sharding**

- Sharding Use cases
- Sharded and Duplicated tables with XMLType columns
- XML operators push down
- XML Search Index

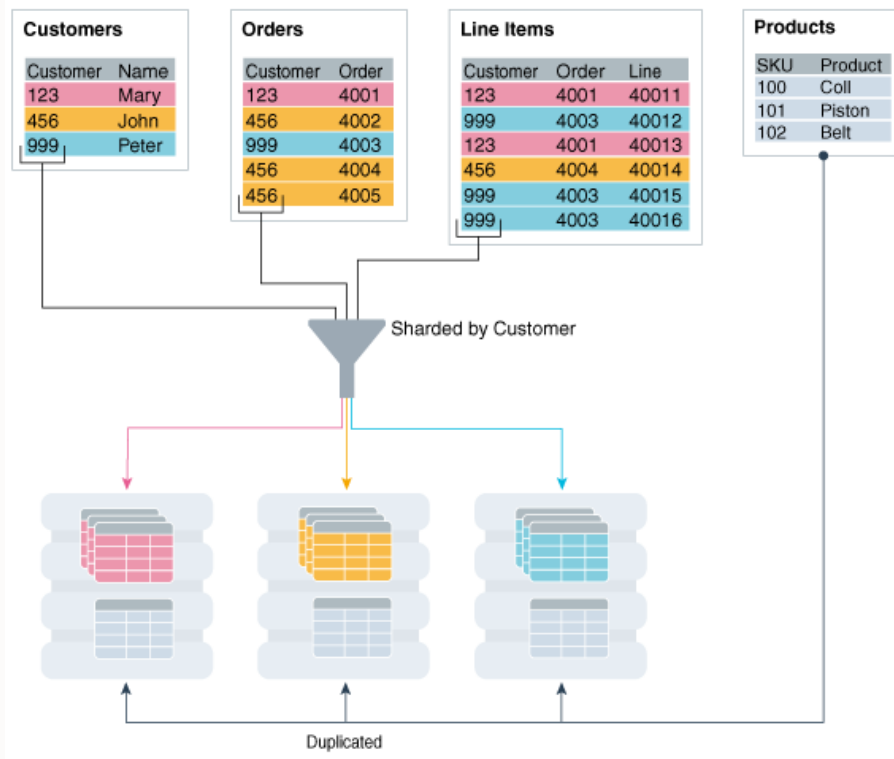


Database Sharding

Horizontal partitioning of a large database into many independent databases (shards)



Database Sharding



Principle of Sharding

- Hyperscale computing solution
- Shared nothing
- Coordinator and shards

Shared Tables

- Information partitioned and balanced across shards using a **sharding key**

Duplicated Tables

- Information replicated across all shards

Sharding Use Cases



Hyperscale

Add shards online to increase database size and throughput. Online elasticity



Fault Isolation

Break the monolith
Fault of one shard has no impact on others



Data Sovereignty

User-defined data placement
for complying with regulatory
requirements

Sharded and duplicated tables with XMLType Table

Sharded Table with XMLType Column

Shard key iso_ctype_cd

```
CREATE SHARDED TABLE purchaseorder (  
  po_id          NUMBER NOT NULL,  
  iso_ctype_cd   VARCHAR2(3) NOT NULL,  
  name           VARCHAR2(50),  
  po_detail      XMLTYPE,  
  CONSTRAINT cust_pk PRIMARY KEY(po_id, iso_ctype_cd))  
PARTITION BY LIST (iso_ctype_cd)  
(  
  PARTITION p_region_1 VALUES ('RG1') TABLESPACE tbs1,  
  PARTITION p_region_2 VALUES('RG2') TABLESPACE tbs6,  
  PARTITION p_region_3 VALUES('RG3') TABLESPACE tbs11  
)  
XMLTYPE COLUMN po_detail STORE AS SECUREFILE TRANSPORTABLE BINARY XML;
```

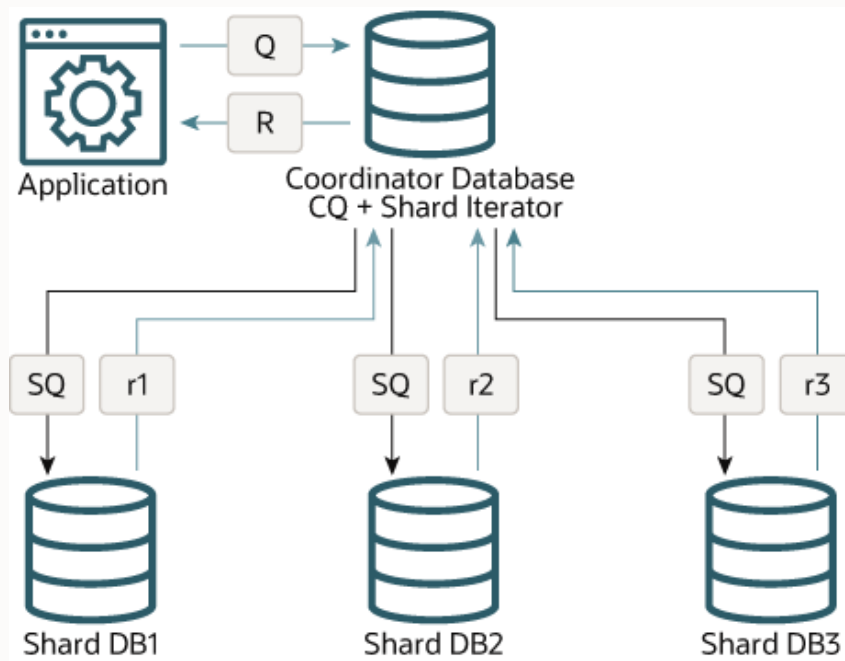
Duplicated Table with XMLType Column

```
CREATE DUPLICATED TABLE purchaseorder_dup (  
    po_id          NUMBER NOT NULL,  
    name           VARCHAR2(50),  
    po_detail      XMLTYPE);  
XMLTYPE COLUMN po_detail STORE AS SECUREFILE TRANSPORTABLE BINARY XML;
```

- Sharded and duplicated tables can only be created in the CATALOG of a sharded DB.
- Sharded tables will split the data across the shards depending on the specified partitions.
- Duplicated tables will replicate the data across all shards.
- DML on sharded and duplicated tables are also available using **direct routing** (connecting directly to a shard).

Pushdown of XML operators

Sharding Push Down



- Q: incoming query
- CQ: Coordinator Query
- SQ: Shard Query
- r1, r2, r3: results from each shard
- R: Final query result

XML Operators Pushed Down



XQuery functions

- XMLExists() : Filtering
- XMLQuery() : Fragment Extraction
- XMLCast() : Conversion to SQL type system
- XMLTable() : Projection

Other functions

- XMLSerialize(): Serializing XML data into a string or LOB
- XMLElement(): Generates a new XML based on a column
- XMLConcat(): Concatenates 2 XML documents

Query Push Down

Shard key iso_ctype_cd

```
SELECT iso_ctype_cd XMLCAST(XMLQUERY (
                                '/PurchaseOrder/Reference'
                                PASSING p.po_detail
                                returning content) AS VARCHAR2(30)) "Reference"
FROM purchaseorder p
WHERE XMLEXISTS (
    '$PO/PurchaseOrder[Reference="SBELL-2023100912333601PDT"]'
    PASSING p.po_detail as "PO"
);
```

- XMLCAST(XMLQUERY()) and XMLEXISTS will be pushed down into a remote query to each shard.
- Each shard is going to run the exact same query issued in the catalog.
- Catalog will just gather the results and show them.



Non-pushable Scenarios

Shard key iso_ctype_cd

```
SELECT iso_ctype_cd
      XMLQUERY ('copy $tmp := $src modify (
                    delete node $tmp/PurchaseOrder/User',
                    insert not $ext/PurchaseOrder/User as lat into $tmp)
                return $tmp'
                PASSING p.po_detail as "src", ext.po_detail as "ext"
                returning content) "UserModified"
FROM purchaseorder p
JOIN external_porder ext
ON p.po_id = ext.po_id;
```

- There are 2 sharded tables involved in the XMLQUERY.
- There is no way to tell in one shard if there are other rows that comply with the JOIN condition.
- XMLQUERY needs to be executed on catalog, unless we add the shard key in the predicate.



Sharding of XML Demo

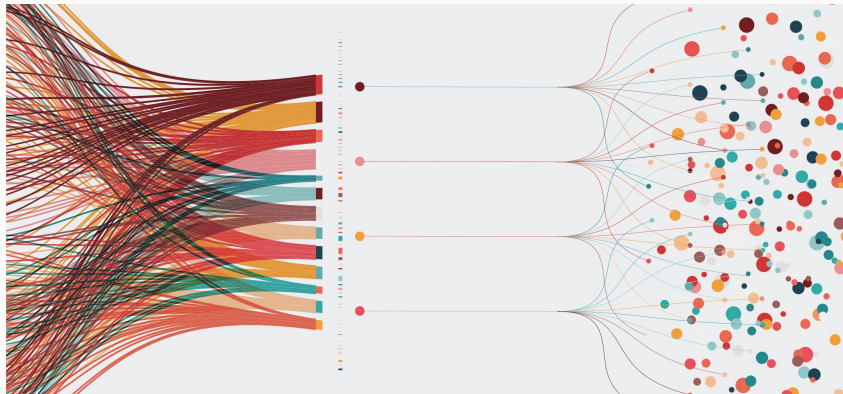


XML Search Index



Why XML Search Indexes?

- Unstructured documents with variable format
- Content and fields are not fixed
- Fields are not mandatory and might be missing from multiple documents.
- Data types are not enforced or there are large text fields.



What do I use an XML Search Index for?

- Full-text search
- Range-search over singleton values
- Data type sub-selection

```
<PurchaseOrder orderDate="1999-12-01" id="1">
  <ShippingInstructions>
    <name>Allan D. McEwen</name>
    <address>Oracle Plaza Redwood Shores</address>
  </ShippingInstructions>
  <LineItems>
    <LineItem ItemNumber="1">
      <Description>Java complete reference</Description>
      <Part Id="2748329425" UnitPrice="10" Quantity="5" />
    </LineItem>
    <LineItem ItemNumber="2">
      <Description>Julius Caesar</Description>
      <Part Id="86471878626" UnitPrice="36.5" Quantity="10" />
    </LineItem>
    <LineItem ItemNumber="3">
      <Part Id="86471878626" UnitPrice="49" Quantity="10" />
    </LineItem>
  </LineItems>
</PurchaseOrder>
```


How to create an XML Search Index

```
CREATE SEARCH INDEX [<schema name>.<index name>  
ON [<schema name>.<table name> (<column name>) FOR XML  
PARAMETERS ('SEARCH_ON <search_on mode>(<datatypes>')');
```

- By default, XQuery Full-Text (XQFT) is enabled
- Used for contains text, ftand, ftor, ftnot, not in
- Search_on mode can be TEXT, VALUE, TEXT_VALUE
- Datatypes can be NUMBER, BINARY_DOUBLE, TIMESTAMP, VARCHAR2
- Allowed only for Transportable Binary XML column

SEARCH_ON TEXT

```
CREATE SEARCH INDEX pidx ON purchaseorder(doc) FOR XML PARAMETERS  
( 'SEARCH_ON TEXT' );
```

OR

```
CREATE SEARCH INDEX pidx ON purchaseorder(doc) FOR XML;
```

```
SELECT id FROM purchaseorder WHERE  
XMLEXISTS(' /PurchaseOrder/LineItems/LineItem/Description[. contains text "Julius"]'  
PASSING doc);
```

- By default, XQuery Full-Text (XQFT) is enabled.
- Used for contains text, ftand, ftor, ftnot, not in

SEARCH_ON VALUE



```
CREATE SEARCH INDEX pidx ON purchaseorder(doc) FOR XML PARAMETERS  
( 'SEARCH_ON VALUE(BINARY_DOUBLE, VARCHAR2)');
```

```
SELECT id FROM purchaseorder WHERE  
XMLEXISTS('/PurchaseOrder/LineItems/LineItem/Part[@UnitPrice > xs:double(20.5)]'  
PASSING doc);
```

```
SELECT id FROM purchaseorder WHERE  
XMLEXISTS('/PurchaseOrder/LineItems/LineItem[Description > "Light"]'  
PASSING doc);
```

- Range-search over scalar predicates
- Type-cast to data type if required
- Index is picked up for predicates using relational operators (>, <, ==, >=, <=, !=).
- Cannot answer full-text queries, if XQuery Full Text operators are present in an XMLEXISTS predicate.

SEARCH_ON TEXT_VALUE



```
CREATE SEARCH INDEX pidx ON purchaseorder(doc) FOR XML PARAMETERS  
( 'SEARCH_ON TEXT_VALUE (BINARY_DOUBLE, TIMESTAMP));
```

```
SELECT id FROM purchaseorder WHERE XMLEXISTS('/PurchaseOrder[@orderDate >  
xs:dateTime("2022-06-23")] ' PASSING doc);
```

```
SELECT id FROM purchaseorder WHERE  
XMLEXISTS('/PurchaseOrder/LineItems/LineItem/Description[. contains text "Julius"] ' PASSING doc);
```

- **Best of both worlds!**
- Can choose between range-search for strings and full-text search or mix both types of predicates



XML Search Index Agenda

- Background
- Syntax
- **Index selection**
- Important considerations
- Index migration
- Demo



Which index is used?

Structured XML Index

Preferred index for range search if both indexes are present

XML Search Index

Used for range-search if

- SXI is not present or
- **ora:use_xmltext_idx** pragma is present

Used for full-text search

NO_INDEX hint can be used to avoid considering the indexes at all

Examples:

```
SELECT count(*) FROM TEST_TBX_NSB t WHERE xmlexists('(# ora:use_xmltext_idx #)
{$row/doc[@cd > xs:date("2020-2-15")]}' passing t.doc as "row");
```

```
SELECT /*+ NO_INDEX(purchaseorder PIDX)*/ id
FROM purchaseorder WHERE xmlexists('/PurchaseOrder/LineItems/LineItem/Part[@UnitPrice > xs:double(500)]'
PASSING doc);
```


Important Considerations when using XML Search Index

The following restrictions need to be considered when writing queries using XML Search Indexes

- XPath steps are only along forward and descendent axes
- You can pass only one XMLType instance as a SQL expression in the PASSING clause of SQL/XML function XMLEExists

When using range-search predicates:

- The comparison value must be a compile time constant or a bind variable that was bound prior to executing the cursor
- The relational predicate must not be !=
- The predicate must not be negated by using the NOT()



Index migration

Oracle recommends XML Search Index for indexing unstructured sections of XML documents

- XML Search Indexes allow for much greater flexibility in data partitioning.

Legacy queries for XMLType stored as non-TBX + SQL contains function

- Replace contains function with xmlexists

Legacy queries for XMLType stored as non-TBX + Context Index

- ora:use_xmltext_idx is only required when structured index is also present

Switch to TBX + XML Search Index. 😊

```
SELECT index_name, index_type, ityp_owner, ityp_name FROM user_indexes;
```

- ityp_name before= 'CONTEXT'; after = 'CONTEXT_V2'

```
SELECT column_name, storage_type FROM user_xml_tab_cols;
```

- storage_type before= 'BINARY'; after = 'TRANSPORTABLE BINARY'

XML Search Index Demo





Summary

Improve your performance. Keep your information in its own region.

1

Store, use, and manage relational data and XML documents in several databases across different geographical regions, acting all like one. Unified management, security, consistency model.

2

Flexible relational and XML access of your data, leveraging Oracle Database's core enterprise features for availability, security, scalability, and performance.

Where to get more information

Oracle.com

- Oracle XML DB - <https://www.oracle.com/database/technologies/appdev/xmlldb.html>
- Oracle Autonomous Database - <https://www.oracle.com/database/autonomous-database.html>

Documentation

- XML DB Developer Guide - <https://docs.oracle.com/en/database/oracle/oracle-database/23/adxdb/index.html>
- Sharding DB Guide – <https://docs.oracle.com/en/database/oracle/oracle-database/19/shard/sharding-overview.html#GUID-60536817-38C5-440A-8404-F2CC810BBF3B>
- Oracle Text Reference - <https://docs.oracle.com/en/database/oracle/oracle-database/23/ccref/index.html>

Livelab

- Oracle XML DB - <https://apexapps.oracle.com/pls/apex/dbpm/r/livelabs/view-workshop?wid=3661>



Thank you



ORACLE



Our mission is to help people see
data in new ways, discover insights,
unlock endless possibilities.

